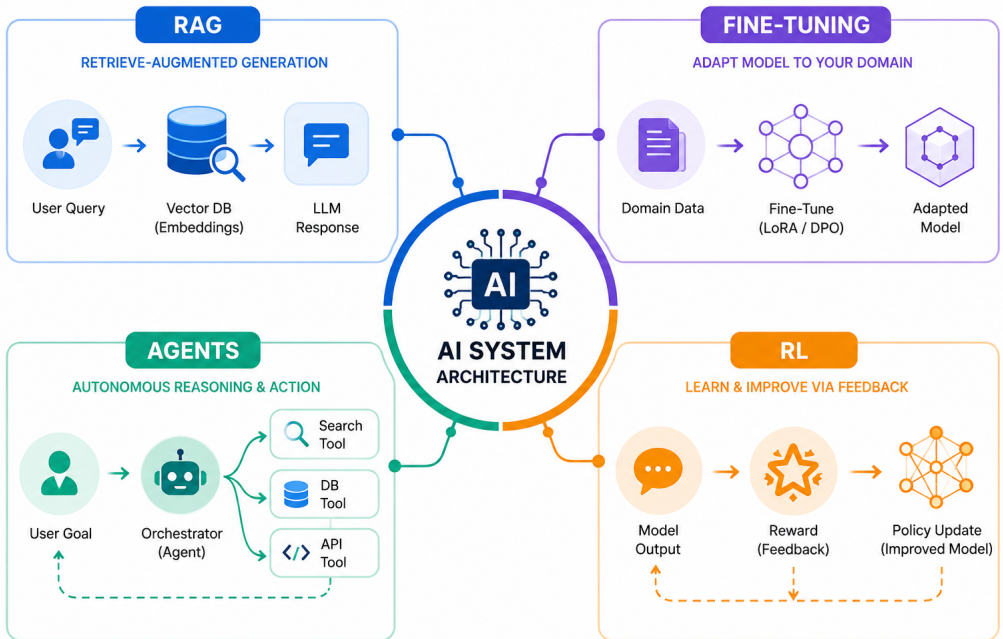




AI System Design Patterns At a Glance

RAG, Agentic RAG, Multi-Agent, and Fine-Tuning: the 4
foundational architectural patterns.



Why Architecture Patterns Matter

Pick the right pattern; do not default to trendy hype

The Cost of Wrong Choices

Every AI system is built from recurring patterns. Picking the wrong one means paying for unnecessary complexity (cost, latency, failure surface) or underbuilding a system that cannot solve the task.

1. RAG Pipeline

Fixed retrieve-then-generate path over external vector databases.

2. Agentic RAG

LLM dynamically decides when, what, and how many times to retrieve.

3. Multi-Agent

Multiple specialized sub-agents coordinated by a central routing supervisor.

4. Fine-Tuning

Baking domain behavior, tone, and language directly into model weights.



Core rule: Complexity should be earned by demonstrated system limitations, never assumed upfront.



AI Coach John

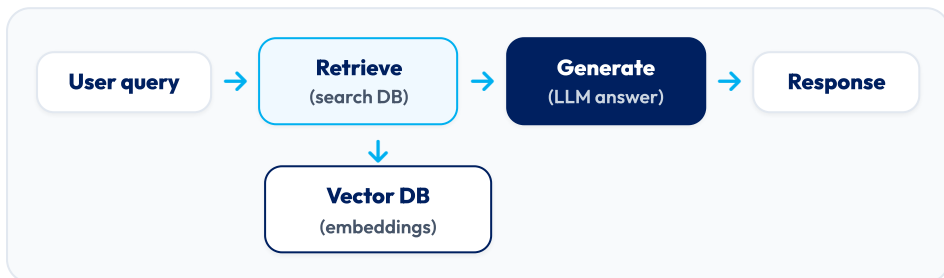


FOLLOW

 Repost

RAG: Fixed Pipeline

Retrieval-Augmented Generation workflow



How It Works

Every query follows the exact same path: embed the query, search the vector store, stuff top results into a prompt, generate an answer. No branching, no decisions, no loops.

Best For

Single-source, single-hop Q&A where queries are fairly consistent (docs search, FAQ bots, internal knowledge bases).



SLA & Tradeoffs: Lowest latency (<800ms) and lowest cost (1 LLM call per query), but fails when questions require multi-hop reasoning.



AI Coach John



FOLLOW

 Repost

Agentic RAG: Dynamic Loops

LLM decides when, what, and how many times to retrieve

↑ **Direct answer, no retrieval needed (Decide → Generate)**

Decide
retrieve or
answer?



Retrieve
search knowledge
base



Grade
check if relevant



Generate
compose answer

↪ **Insufficient context: retry with new query (Grade → Retrieve)**

How It Works

A decision node first checks if retrieval is even needed. If yes, it retrieves, then grades what came back: if it is not good enough, it loops back and tries a different search instead of blindly generating from weak context.

Best For

Mixed query types, multi-hop questions, or any system where a fixed pipeline would either over-retrieve (waste) or under-retrieve (hallucinate).



Production Guardrail: Always enforce a strict retry limit (max 2-3 loops) to prevent runaway latency and excessive token costs.



AI Coach John

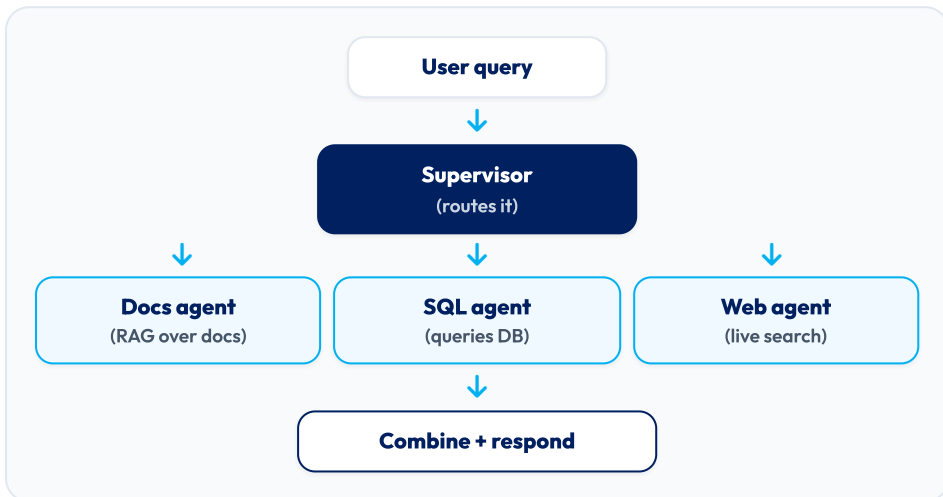


FOLLOW

 **Repost**

Multi-Agent: Routing

A supervisor routing to specialized domain agents



How It Works

The supervisor reads the question and routes it to one or more specialized sub-agents, each an expert in one domain, source, or tool. Results get combined into a single final answer.



Best For: Systems spanning genuinely distinct domains (internal docs + SQL warehouse + live APIs) where a single monolithic agent breaks down.





PATTERN 04 WORKFLOW

Fine-Tuning: Pipeline

Baking behavior into the model itself

Collect data
(examples)



Label / curate
dataset



Train
(LoRA/DPO)



Evaluate
(benchmark)



Deploy
(serve model)

🔄 Iterate: if eval does not improve, adjust data/params & retrain (Evaluate → Train)



How It Works

Instead of shaping behavior through prompts or retrieval at inference time, you train the change directly into model weights using curated examples, evaluating against a baseline before deploying.



Best For

Consistent behavior, tone, or format you want baked in permanently, domain-specific language the base model does not know, or when prompting and RAG have plateaued.



Rule of Thumb: Fine-tuning teaches form, style, and jargon; RAG injects real-time dynamic knowledge.



AI Coach John



FOLLOW

🔄 Repost

Side-by-Side Comparison

All 4 patterns, one comprehensive table

PATTERN	COMPLEXITY	LATENCY	COST / QUERY	ADAPTABILITY
RAG	Low	Low	Low (1 LLM call)	Low: fixed path
Agentic RAG	Med-High	Med-High	Medium (3 to 6 calls)	High: decides & loops
Multi-Agent	High	High	High (multi-agent)	High: specialized routing
Fine-Tuning	High upfront, low runtime	Low runtime	Low runtime, high train cost	Low: needs retrain to change

✓ Architectural Fit Summary

RAG: Single-source, single-hop Q&A over static/dynamic documents.

Agentic RAG: Mixed or multi-hop queries requiring self-correction.

Multi-Agent: Enterprise platforms spanning SQL, APIs & documents.

Fine-Tuning: Permanent industry vocabulary, tone, and compliance baking.



Selection SLA: Always start with the simplest architecture that meets your latency and accuracy requirements.



AI Coach John



FOLLOW

 Repost

Which Pattern Should You Build?

Progressive architecture selection roadmap

▶ **Start with RAG**

If queries are consistent and single-hop over one data source. It is the cheapest, fastest, and easiest to debug.

↑ **Move to Agentic RAG**

If you see wasted retrievals on simple questions, or failures on multi-hop questions a fixed pipeline cannot handle.

🧩 **Move to Multi-Agent**

If your system genuinely spans multiple distinct domains or sources that each need different tools, prompts, or logic (not just multiple data sources under one retriever).

⚙️ **Add Fine-Tuning**

If you have exhausted prompting and RAG and still need consistent tone, domain-specific language, or behavior that prompting cannot reliably enforce.

⚠️ Common mistake: Jumping straight to the most complex pattern because it sounds more advanced. Earn complexity by demonstrated limitations.



Combining Patterns In Production

Real production systems mix and match

Patterns Are Not Mutually Exclusive

- A **multi-agent** system where each sub-agent internally runs **agentic RAG**.
- A **fine-tuned** base model used inside a **RAG** pipeline (fine-tuned for tone + grounded via retrieval for facts).
- An **agentic RAG** system where the final generation step uses a **fine-tuned** model for consistent formatting.

Supervisor



Docs agent

(uses agentic RAG loop)



Fine-tuned

model output

Enterprise Deployment Synergy

By decoupling routing (Multi-Agent), dynamic reasoning (Agentic RAG), and style compliance (Fine-Tuning), you achieve high accuracy without prompt overflow.



Takeaway: Think of these as composable building blocks, not competing architectures. Pick the combination that matches where each part needs adaptability, grounding, or specialization.



AI Coach John



FOLLOW

 Repost

4 Patterns, One Decision to Make

RAG, Agentic RAG, Multi-Agent, and Fine-Tuning summary

1. RAG

RAG for Simplicity: Single-source retrieve-then-generate pipeline.

2. Agentic RAG

Agentic RAG for Adaptability: Dynamic retrieval & self-correction loops.

3. Multi-Agent

Multi-Agent for Specialization: Routing across distinct domains.

4. Fine-Tuning

Fine-Tuning for Permanence: Baking behavior into model weights.

✓ Production Reality

Most systems end up using more than one pattern to combine factual grounding, dynamic reasoning, and permanent formatting.

🔗 **Action:** Save this as your AI system architecture reference. Follow for more AI engineering breakdowns.



AI Coach John



FOLLOW

🔄 Repost